Tunnijuhend: React räägib serveriga

11IT · Veebirakendused · 8. tund · tunnis

Täna võtad söökla rakendusest näidisandmed ära ja leht hakkab arvustusi serverilt küsima.

> **Vaja läheb:** Node.js, npm, VS Code ja söökla projekt või kursuse lehelt algfailid.
>
> **Kuidas töötada:** ennusta, mis juhtub → kirjuta kood → käivita → selgita, mis muutus. Kirjuta kood ise, nii märkad, mis mida teeb. Vigade otsimiseks on kaks kohta: brauseri vead on konsoolis (`F12`), serveri vead on selle terminalis.
>
> **Miinimum:** sammud 0–4, leht näitab serverist tulnud arvustusi. Sammud 5–8 teevad rakenduse töökindlaks.

Samm 0: projekt lahti

Ava söökla projekt VS Code'is: File → Open Folder ja vali kaust, kus on failid `package.json` ja `vite.config.ts`. Seda kausta nimetame edaspidi **projekti peakaustaks**. **Kui sul projekti pole** või see ei tööta, laadi kursuse lehelt alla algfailid `tunnitoo-react-server.zip`, paki lahti, ava kaust `sookla` ja tee terminalis:

```
npm install
cd server
npm install
cd ..
```

`npm install` paigaldab paketid, mida projekt vajab. Seda on vaja teha kaks korda, sest lehel ja serveril on kummalgi oma `package.json`. `cd` tähendab „mine kausta" ja `cd ..` „mine üks kaust ülespoole".

Samm 1: pane server tööle

Ava terminal (Terminal → New Terminal). Käsk `pwd` näitab, mis kaustas oled: see peab olema projekti peakaust. Siis kirjuta:

```
cd server
node index.js
```

Nüüd peaksid nägema: terminalis `Server töötab: http://localhost:3000` ja uut käsurida ei tule. See on õige, server ootab. Ava brauseris `http://localhost:3000/api/arvustused`: seal on kolm arvustust JSON-ina.

Kui näed midagi muud: `Cannot find module` tähendab, et sa pole kaustas `server`, fail `index.js` puudub või paketid on paigaldamata. Kontrolli `pwd`-ga kausta ja `ls`-iga faile. `EADDRINUSE` tähendab, et port 3000 on juba mõne teise programmi käes. See võib olla sama server teises terminalis.

Samm 2: pane leht tööle teises terminalis

Server hõivab oma terminali, nii et vaja on teist. Vajuta terminali paneelis nuppu **+**. Kontrolli `pwd`-ga, et oled projekti peakaustas. Kui ei ole, mine sinna `cd`-ga. Kirjuta:

```
npm run dev
```

Nüüd peaksid nägema: terminalis aadress, tavaliselt `http://localhost:5173/`. Ava see aadress, mille Vite **kirjutas**: seal on sinu söökla leht, nagu enne. Nüüd töötab korraga kaks programmi. Terminalide vahel saad paneeli paremas servas liikuda.

Kui näed midagi muud: kui `npm run dev` annab vea `Missing script`, oled kaustas `server`. Mine `cd ..`-ga tagasi.

Samm 3: vahendusserver

Nüüd õpetame Vite'ile, et kõik `/api`-ga algavad päringud lähevad edasi serverile. Ava projekti peakaustas fail `vite.config.ts`. Lisa seadistusobjektis omaduse `plugins` järele omadus `server`, nagu järgmises näites.

`vite.config.ts`

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:3000',
    },
  },
})
```

Kood	Mida see teeb
`server: {`	Vite'i arendusserveri seaded.
`proxy: {`	Vahendusserveri seaded: millised päringud edasi saata.
`'/api': 'http://localhost:3000',`	Kui aadress algab /api-ga, saada edasi Expressile pordil 3000.

Mis sinu arvates juhtub, kui avad pärast salvestamist Vite'i pordil aadressi `/api/arvustused`? Salvesta. Vite märkab muutust ja käivitub ise uuesti.

Nüüd peaksid nägema: ava `http://localhost:5173/api/arvustused`. Ometi on see Vite'i port ja näed serveri arvustusi! Vahendusserver töötab.

Kui näed midagi muud: lehekülg on tühi või näed Reacti lehte: kontrolli sulge ja koma failis `vite.config.ts` ning käivita Vite uuesti (`Ctrl+C`, siis `npm run dev`). Veateate korral vaata, kas server töötab ja mida Vite'i terminal ütleb.

Samm 4: küsi arvustused serverilt

Ava `src/App.tsx`. Teeme kolm muudatust.

Esiteks muuda faili ülaosas importe: lisa `useEffect` ja kustuta impordirida, millega tuuakse sisse `naidisArvustused`.

```
import { useEffect, useState } from 'react';
```

Teiseks muuda esimest `useState`-rida, nii et alguses oleks nimekiri tühi:

```
const [arvustused, setArvustused] = useState<Arvustus[]>([]);
```

Kolmandaks lisa kohe viimase `useState`-rea järele see kood:

```
  async function laeArvustused() {
    const vastus = await fetch('/api/arvustused');
    const andmed: Arvustus[] = await vastus.json();
    setArvustused(andmed);
  }

  useEffect(() => {
    laeArvustused();
  }, []);
```

Kood	Mida see teeb
`async function laeArvustused() {`	Funktsioon, mis toob andmed. async lubab kasutada await-i.
`await fetch('/api/arvustused')`	Saadab päringu ja kasutab tulemust, kui see on kohal. Aadress algab /api-ga, nii et Vite saadab selle serverile.
`await vastus.json()`	Loeb JSON-vastuse JavaScripti massiiviks.
`setArvustused(andmed);`	Kirjutab massiivi olekusse. React joonistab lehe uuesti.
`useEffect(() => {`	Käivita see funktsioon pärast seda, kui leht on joonistatud.
`}, []);`	Tühi sõltuvuste massiiv: mitte iga joonistamise järel.

Salvesta ja kustuta fail `src/andmed.ts` (paremklõps failil → Delete). Seda pole enam vaja.

Nüüd peaksid nägema: lehel on endiselt kolm arvustust, aga nüüd tulevad need serverist. Kontrolli: muuda failis `server/index.js` ühe arvustuse kommentaari, salvesta, pane server `Ctrl+C`-ga kinni, käivita uuesti `node index.js` ja värskenda lehte (`F5`). Uus tekst on lehel. **Miks?** Selgita pinginaabrile.

Kui näed midagi muud: tühi leht: vaata konsooli (`F12`). Tabel levinud vigadega on konspekti lõpus.

> **Vaata ise, kuidas päring käib.** Vajuta `F12`, vali vahekaart **Network** ja värskenda lehte. Otsi rida `arvustused` ja klõpsa sellel. Vahekaardil **Response** on täpselt see, mille server saatis. Ridu on kaks, sest projektis on sisse lülitatud StrictMode, mis arenduse ajal käivitab efekti alguses kaks korda.

Samm 5: laadimine ja vead

Nüüd tuleb kasutajale öelda, mis toimub. Lisa teiste `useState`-ridade juurde kaks uut:

```
const [laeb, setLaeb] = useState(true);
const [viga, setViga] = useState('');
```

`laeb` on `true` ehk tõene, kuni andmed pole veel kohal. `viga` on alguses tühi tekst. Muuda funktsioon `laeArvustused` selliseks:

```
async function laeArvustused() {
  try {
    const vastus = await fetch('/api/arvustused');
    if (!vastus.ok) {
      throw new Error('Server vastas veaga');
    }
    const andmed: Arvustus[] = await vastus.json();
    setArvustused(andmed);
  } catch {
    setViga('Arvustusi ei saanud kätte. Kas server töötab?');
  }
  setLaeb(false);
}
```

Kood	Mida see teeb
`try {`	Proovi teha seda, mis on järgnevates sulgudes.
`if (!vastus.ok) {`	Kui server ei vastanud õnnestunult. ! tähendab „ei“.
`throw new Error('Server vastas veaga');`	Tekita viga: programm hüppab kohe catch-i.
`} catch {`	Siia jõuab programm, kui try sees läks midagi valesti.
`setViga('Arvustusi ei saanud...');`	Kirjutab veateksti olekusse.
`setLaeb(false);`	Laadimine on läbi, hoolimata tulemusest.

Lõpuks näita teateid. Pane need JSX-is kohe rea `</form>` järele:

```
{laeb && <p className="kokku">Laen arvustusi…</p>}
{viga && <p className="kokku">{viga}</p>}
```

Nüüd peaksid nägema: leht töötab nagu enne. „Laen arvustusi…“ on nii lühidalt, et sa ei pruugi seda märgatagi.

Samm 6: tee meelega katki

Mis sinu arvates juhtub, kui server on kinni ja värskendad lehte? Mine serveri terminali, vajuta `Ctrl+C` ja värskenda lehte (`F5`).

Nüüd peaksid nägema: arvustusi pole ja lehel on teade „Arvustusi ei saanud kätte. Kas server töötab?“. Täpselt nii peabki olema! Vaata ka Vite’i terminali ja F12 all vahekaarti Network: mis veateade ja staatus seal on? **Miks see juhtus?**

Käivita server uuesti (`node index.js`) ja värskenda lehte. Arvustused on tagasi.

Samm 7: mida server veel ei oska

Lisa lehel vormiga uus arvustus. See ilmub nimekirja. **Mis sinu arvates juhtub**, kui vajutad `F5`? Vajuta.

Nüüd peaksid nägema: sinu arvustus on kadunud, algsed kolm on alles. Vormi kood lisab arvustuse ainult sinu brauseri mällu ja server ei kuulnud sellest midagi. Järgmises tunnis õpetame serveri arvustusi vastu võtma.

Samm 8: salvesta Gitti

Git on programm, mis hoiab alles koodi ajalugu. Mine terminali, kus ükski server ei tööta. Vajadusel vajuta jälle **+**. Kontrolli, et oled projekti peakaustas, ja tee:

```
git add .
git commit -m "React küsib arvustused serverilt"
```

`git add .` valib kõik muudatused ja `git commit` salvestab need koos sõnumiga. Kaks võimalikku takistust: ZIP-failist võetud projektil pole Giti ajalugu, nii et kui Git ütleb `not a git repository`, tee enne seda üks kord `git init`. Kui Git küsib nime ja e-posti, täida need käskudega:

```
git config --global user.name "Sinu Nimi"
git config --global user.email "sinu@naide.ee"
```

Kogu fail

Kui midagi ei tööta, võrdle oma faili sellega ja otsi, kus on erinevus.

`src/App.tsx`

```
import { useEffect, useState } from 'react';
import type { FormEvent } from 'react';
import type { Arvustus } from './types';
import ArvustuseKaart from './ArvustuseKaart';
import './App.css';

export default function App() {
  const [arvustused, setArvustused] = useState<Arvustus[]>([]);
  const [laeb, setLaeb] = useState(true);
  const [viga, setViga] = useState('');
  const [toit, setToit] = useState('');
  const [hinne, setHinne] = useState(5);
  const [kommentaar, setKommentaar] = useState('');

  async function laeArvustused() {
    try {
      const vastus = await fetch('/api/arvustused');
      if (!vastus.ok) {
        throw new Error('Server vastas veaga');
      }
      const andmed: Arvustus[] = await vastus.json();
      setArvustused(andmed);
    } catch {
      setViga('Arvustusi ei saanud kätte. Kas server töötab?');
    }
    setLaeb(false);
  }

  useEffect(() => {
    laeArvustused();
  }, []);

  function lisa(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    const uus: Arvustus = { id: Date.now(), toit, hinne, kommentaar };
    setArvustused([uus, ...arvustused]);
    setToit('');
    setHinne(5);
    setKommentaar('');
  }

  return (
    <main>
      <h1>Söökla arvustused</h1>
```

```
      <form onSubmit={lisa}>
        <input value={toit} onChange={(e) => setToit(e.target.value)} placeholder="Mis toit?" />
        <select value={hinne} onChange={(e) => setHinne(Number(e.target.value))}>
          <option value={5}>5 ★</option>
          <option value={4}>4 ★</option>
          <option value={3}>3 ★</option>
          <option value={2}>2 ★</option>
          <option value={1}>1 ★</option>
        </select>
        <input
          value={kommentaar}
          onChange={(e) => setKommentaar(e.target.value)}
          placeholder="Kommentaar"
        />
        <button type="submit">Lisa</button>
      </form>

      {laeb && <p className="kokku">Laen arvustusi…</p>}
      {viga && <p className="kokku">{viga}</p>}

      <ul>
        {arvustused.map((a) => (
          <ArvustuseKaart key={a.id} arvustus={a} />
        ))}
      </ul>
    </main>
  );
}
```

Kui jõuad varem valmis

Need on valikulised väljakutsed. Koodi pole, ainult vihjed.

- **Menüü lehel.** Küsi serverilt ka `/api/menyy` ja näita toite nimekirjana. Vihje: uus `useState`, teine funktsioon ja sama `useEffect`.
- **Mitu arvustust?** Näita nimekirja kohal, mitu arvustust on. Vihje: `arvustused.length` ja klass `kokku`.
- **Värskenda-nupp.** Nupp, mis küsib andmed uuesti. Vihje: nupp vajab funktsiooni, mis enne küsimist teeb `setViga('')` ja `setLaeb(true)`, muidu jääb vana veateade alles.

> **Hoia projekt alles!** Järgmises tunnis õpib server arvustusi lisama ja kustutama.